
Introduction to Large Language Models

Week 4 • Lecture 3

Bidirectional Encoder Representations from Transformers (BERT)

Detailed study notes built from the lecture slides of Prof. Mitesh M. Khapra
(AI4Bharat, IIT Madras)

What these notes cover

- **Motivation:** encoder-only, decoder-only and encoder-decoder models, and why bidirectional context matters.
- **Module 3.1, Masked Language Modelling:** why attention masking fails, the [MASK] token, the MLM loss, the 15% rule, the 80/10/10 rule, Next Sentence Prediction, token, segment and position embeddings, pre-training setup, and a full parameter count derivation.
- **Module 3.2, Adapting to downstream tasks:** feature extraction vs fine-tuning, classification heads, and extractive question answering with start and end vectors.
- **Practice:** numerical problems with worked solutions, subjective questions with model answers, and MCQs with explanations.

Contents

1	Where BERT fits: the three transformer families	2
1.1	Recap: three ways to use the transformer	2
1.2	GPT learns left to right. Why is that a problem?	2
2	Module 3.1: Masked Language Modelling (MLM)	3
2.1	We need both directions to predict a masked word	3
2.2	Casting MLM as a probability problem	4
2.3	Attempt 1: mask the attention weights, as in causal LM?	4
2.4	Attempt 2 (the right one): use a special [MASK] token	6
2.5	The MLM loss	6
2.6	BERT: the architecture	8
2.7	Next Sentence Prediction (NSP)	10
2.8	Putting MLM and NSP together	11
2.9	Pre-training setup	12
2.10	Parameter calculation (BERT-base)	13
3	Module 3.2: Adapting BERT to downstream tasks	15
3.1	Classification, feature-based approach	15
3.2	Classification, fine-tuning approach	16
3.3	Extractive Question Answering	17
3.4	Other downstream tasks (brief)	20
4	Quick-revision cheat sheet	22
5	Numerical problems with worked solutions	22
6	Subjective questions with model answers	26
7	Multiple-choice questions	29

1 Where BERT fits: the three transformer families

1.1 Recap: three ways to use the transformer

The original transformer (Vaswani et al., 2017) had two halves: an **encoder** that reads the whole input and a **decoder** that generates the output one token at a time. Each half can be used on its own, so language models come in three families:

1. **Encoder-only models** (e.g. BERT, RoBERTa). Every token can attend to every other token, so they are *bidirectional*. Their natural training task is “fill in the blank”: predict $P(\langle \text{mask} \rangle)$ from both sides of the context.
2. **Decoder-only models** (e.g. GPT). Each token can attend only to the tokens before it (*causal* or *masked* self-attention). They predict $P(x_i | x_1, \dots, x_{i-1})$, which is exactly what you need to generate text.
3. **Encoder-decoder models** (e.g. T5, BART). The encoder reads a corrupted input bidirectionally, and the decoder generates the target, looking at the encoder output through *cross-attention*.

The previous lecture built a language model from the **decoder** (GPT). This lecture uses the **encoder** instead, and the result is **BERT: Bidirectional Encoder Representations from Transformers**.

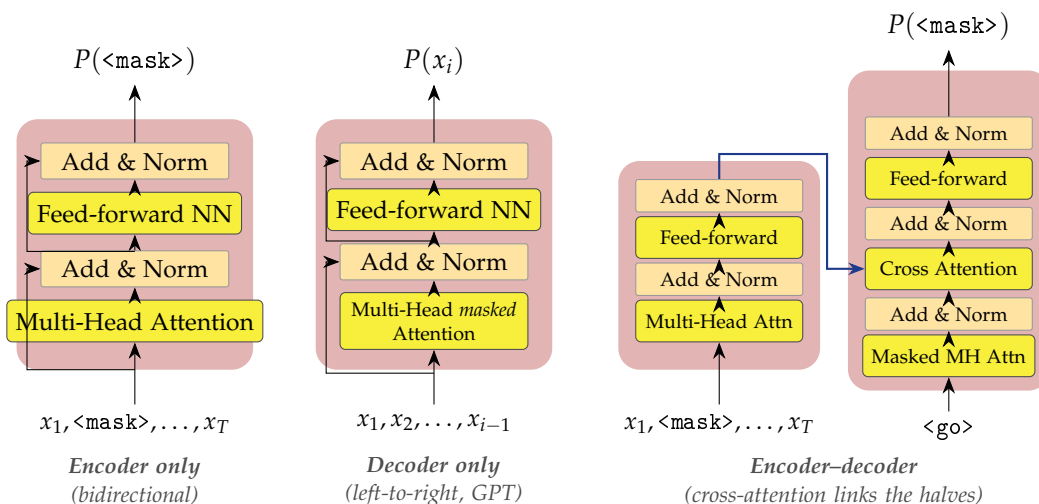


Figure 1: The three transformer families. This week is about the left one.

1.2 GPT learns left to right. Why is that a problem?

In GPT, a word's representation is built only from the words to its *left*. Given “*i like to ____*”, the model scores continuations such as *play, watch, go, read*. That is a natural fit for **text generation**, because at generation time the future really does not exist yet.

Many language-understanding tasks need context from *both* sides:

- **Named Entity Recognition (NER).** In “*Nothing has shipped its new OS to Nothing Phone 2*”, the first *Nothing* is a **company**, not the English word “nothing”. You only know that from what comes *after* it: “has shipped its new OS ... Phone 2”. A left-to-right model reading the first word has seen nothing to its left at all.

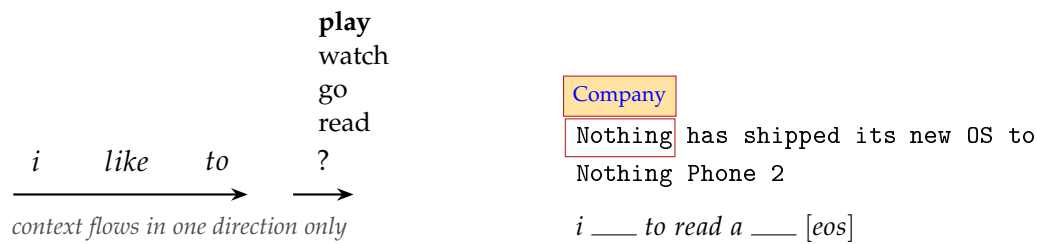


Figure 2: Left: GPT's left-to-right view. Right: tasks where the right-hand context matters.

- **Fill in the blanks.** "*i* ____ to read a ____ [eos]". The first blank ("like", "love") depends on "to read" on its right. The second blank ("book") depends on "read a" on its left and on the end of the sentence.

Intuition : why bidirectionality matters

A left-to-right model is like reading a sentence through a slit that shows only the words you have already passed. For *generating* text that is fine, because you must commit word by word anyway. For *understanding* a sentence you already have in full (classifying it, tagging entities, answering questions about it), covering the right half is an artificial handicap. BERT removes it: every token's representation is built from the entire sentence.

Key idea

Decoder (causal) = the natural choice for generation. Encoder (bidirectional) = the natural choice for understanding. BERT is an encoder-only language model that learns deep bidirectional representations from unlabelled text and is then adapted to downstream understanding tasks.

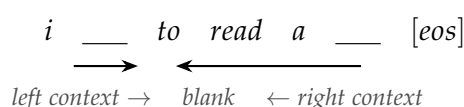
Beyond the slides : why not just train two GPTs, one in each direction?

ELMo (2018) did something similar: a left-to-right LSTM and a right-to-left LSTM, with their outputs concatenated. That is only *shallowly* bidirectional, because the two directions never interact inside the network. At layer ℓ , the left model's representation of token i has never seen the right context. In BERT's self-attention, left and right context are mixed *at every layer*, which the BERT paper calls *deeply bidirectional*.

2 Module 3.1: Masked Language Modelling (MLM)

2.1 We need both directions to predict a masked word

Look again at "*i* ____ to read a ____ [eos]". To fill the first blank we use the word on its left (*i*) and the words on its right (*to read a*). Predicting a word from the words around it is exactly the idea behind **CBOW** (Continuous Bag of Words) in word2vec, which predicts a centre word from a window of surrounding words.



This task is called **Masked Language Modelling (MLM)**. The paper calls it a "Cloze" task, after the fill-in-the-blank reading tests.

Common pitfall / exam trap : why can't we use the decoder?

The decoder's masked (causal) self-attention is *built* to be unidirectional: position i cannot see positions $> i$. If we fed it " i [MASK] to read", the [MASK] position could not see "to read", which is the very context we need. So for MLM we use the **encoder**, whose self-attention has no causal mask.

2.2 Casting MLM as a probability problem

Using the encoder, the problem becomes

Formula : the MLM objective for one masked position

$$P(y_i \mid x_1, \dots, x_{i-1}, x_i = [\text{MASK}], x_{i+1}, \dots, x_T) = ?$$

where y_i is the *original* token at position i , and the model sees the whole corrupted sequence $\tilde{\mathbf{x}}$ (with [MASK] in place of the masked words).

The independence assumption. If several positions are masked (the set $\mathcal{M}$), BERT predicts each one *separately*, as if the masked tokens were independent of each other given the corrupted input:

$$P(\{y_i\}_{i \in \mathcal{M}} \mid \tilde{\mathbf{x}}) \approx \prod_{i \in \mathcal{M}} P(y_i \mid \tilde{\mathbf{x}}).$$

Compare this with the *exact* chain-rule factorisation a causal LM uses:

$$P(x_1, \dots, x_T) = \prod_{i=1}^T P(x_i \mid x_{<i}).$$

The causal LM factorisation holds for any distribution. BERT's product over $\mathcal{M}$ is an *approximation*.

Intuition : what the independence assumption loses

Suppose the input is "[MASK] [MASK] is a city in the US" and the truth is "New York". BERT predicts position 1 and position 2 independently, so it might say "New" for position 1 and "Angeles" for position 2. Each is plausible on its own ("New Delhi", "Los Angeles"), but together they are inconsistent. The two predictions never talk to each other. For *representation learning* this rarely matters. It is one reason BERT is not a good *text generator*. (XLNet's "permutation LM" was designed to fix this.)

The remaining question is **how to mask the words in the input**.

2.3 Attempt 1: mask the attention weights, as in causal LM?

Recall that in a self-attention layer every word attends to every other word. Take the 5-word sentence "*I enjoyed the movie transformers*", with toy 3-d input vectors, and suppose we want to hide "I" (position 1) and "movie" (position 4).

The causal LM hid the future by adding $-\infty$ to the attention scores before the softmax. Can we hide the masked words the same way, by blocking the *columns* of the masked positions?

The answer is no. The lecture's reason: *we want the model to learn, by attending, what the blanks are*. Here is what that means, and a second, deeper reason.

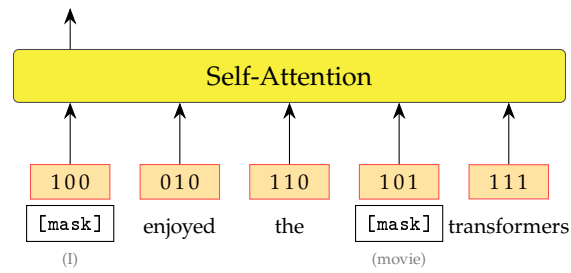


Figure 3: Goal of MLM: hide a few words at random (here “I” and “movie”) and predict them.

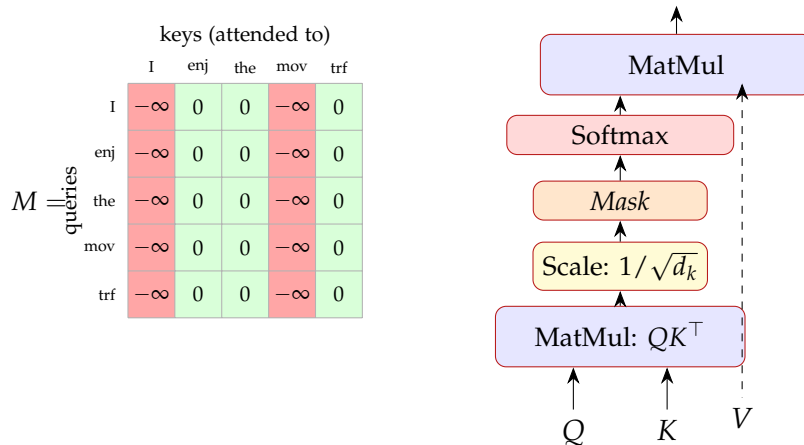


Figure 4: The tempting (but wrong) idea: put $-\infty$ in the columns of the masked words (1 and 4), so that no query can attend to them.

1. **We want to train a representation at the blank.** The prediction for the blank is read off the hidden state at the blank’s position. That position must be able to gather evidence from the rest of the sentence, and the blank itself should be an actual input the model processes, not a hole in the attention pattern. Column masking only stops *others* from looking at the blank. It says nothing about what the blank’s own position “is”.
2. **Column masking would leak the answer.** This is the deeper reason. Even if no query attends to position 4, the *input vector* of position 4 is still the *true word* “movie”. Through the **residual connection** ($h = x + \text{Attention}(x)$), the output at position 4 contains “movie” directly. The model can just copy its own input to its output and get zero loss while learning nothing. In a causal LM this doesn’t happen, because the output at position i predicts the *next* token x_{i+1} , never its own input.

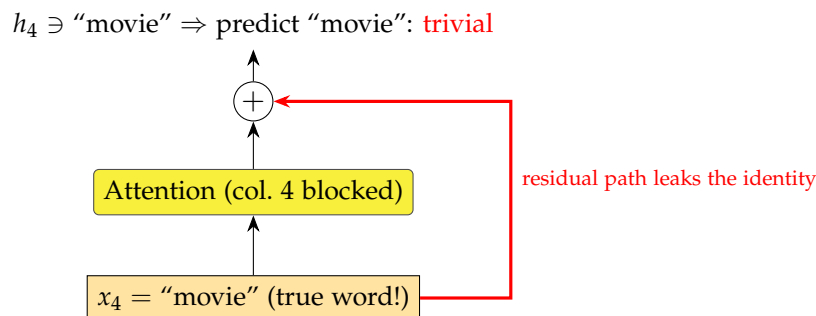


Figure 5: Why masking only the attention columns fails: the residual stream carries the true token to its own output.

Key idea : corrupt the input, not the attention

We must **corrupt the input itself**: replace the chosen word with a special token, so the true identity is never present anywhere in the network. Think of the [MASK] token as **noise that corrupts the original input text**. The model's job is to **recover the original tokens**. This is the same objective as a **denoising autoencoder**: corrupt $x \rightarrow \tilde{x}$, then reconstruct x from $\tilde{x}$. For this reason MLM is also called a **pre-training denoising objective**.

2.4 Attempt 2 (the right one): use a special [MASK] token

Add [MASK] as a **special token in the vocabulary** and learn an embedding for it, like any other word. Then replace the words to be masked with [MASK] *before* the input enters the network. Inside self-attention, [MASK] behaves like an ordinary token: it has its own query, key and value.

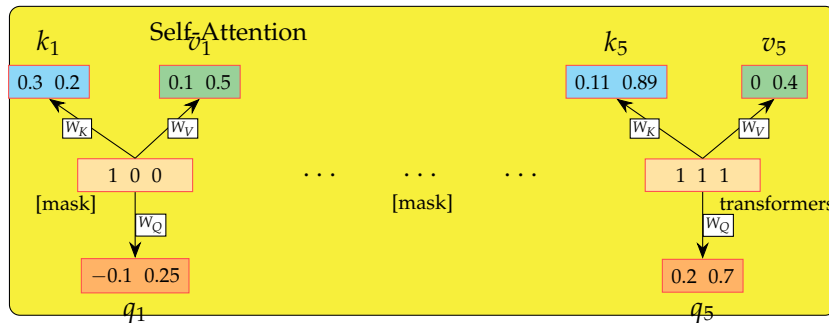


Figure 6: [MASK] is an ordinary vocabulary item with its own learnable embedding. It gets a query, key and value like any word.

Intuition : what does the [MASK] embedding learn?

The [MASK] embedding is the same at every masked position, so on its own it carries *no* information about which word was hidden. Its query $q = W_Q e_{\text{[MASK]}}$ effectively learns to say “I am a hole, show me whatever helps fill me”. After a few layers, the hidden state at that position is almost entirely built from context, and that is the representation we want.

Self-attention recap, with dimensions

Let $\tilde{X} \in \mathbb{R}^{T \times d}$ be the (corrupted) input embeddings, one row per token. For one head with key dimension d_k :

$$Q = \tilde{X}W_Q [T \times d_k], \quad K = \tilde{X}W_K [T \times d_k], \quad V = \tilde{X}W_V [T \times d_v],$$

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) [T \times T], \quad Z = AV [T \times d_v].$$

For BERT's MLM the mask M is **all zeros**, apart from $-\infty$ on [PAD] columns so that padding is ignored. Every token attends to every real token. (The slides write $Q^T K$ because they store vectors as columns. It is the same computation.)

2.5 The MLM loss

Only the set $\mathcal{M}$ of masked positions is predicted, so the cross-entropy loss is computed *only* at those positions.

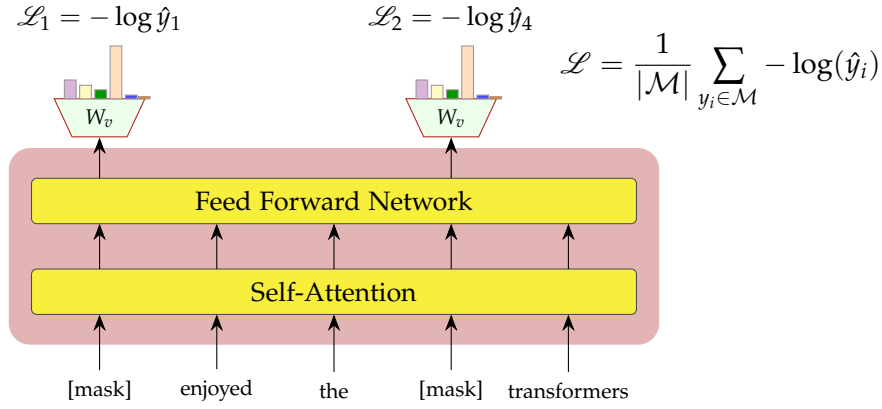


Figure 7: The MLM head is applied only at masked positions (1 and 4). The other outputs are computed but not used in the loss.

Formula : MLM loss, with shapes

For a masked position $i \in \mathcal{M}$ with final hidden state $h_i \in \mathbb{R}^d$ ($d = 768$ for BERT-base):

$$\begin{aligned}
 z_i &= W_v h_i + b & z_i &\in \mathbb{R}^{|\mathcal{V}|}, W_v \in \mathbb{R}^{|\mathcal{V}| \times d} \\
 \hat{\mathbf{p}}_i &= \text{softmax}(z_i) & \hat{\mathbf{p}}_i &\in \mathbb{R}^{|\mathcal{V}|}, \sum_w \hat{p}_{i,w} = 1 \\
 \hat{y}_i &= \hat{p}_{i,y_i} & &\text{probability given to the true token } y_i \\
 \mathcal{L}_{\text{MLM}} &= \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} -\log \hat{y}_i.
 \end{aligned}$$

Derivation: why $-\log \hat{y}_i$ is cross-entropy. The target at position i is the one-hot vector $\mathbf{t}_i$ with $t_{i,y_i} = 1$. The cross-entropy between the target and the prediction is

$$H(\mathbf{t}_i, \hat{\mathbf{p}}_i) = - \sum_{w \in \mathcal{V}} t_{i,w} \log \hat{p}_{i,w} = -\log \hat{p}_{i,y_i} = -\log \hat{y}_i,$$

because every term except $w = y_i$ is multiplied by 0. Averaging over masked positions gives $\mathcal{L}_{\text{MLM}}$. Minimising it is the same as maximising $\sum_{i \in \mathcal{M}} \log P(y_i | \bar{\mathbf{x}})$, the log-likelihood under the independence assumption above.

The gradient, and why only masked positions learn. With softmax plus cross-entropy, the gradient with respect to the logits is the familiar

$$\frac{\partial(-\log \hat{y}_i)}{\partial z_i} = \hat{\mathbf{p}}_i - \mathbf{t}_i \in \mathbb{R}^{|\mathcal{V}|}.$$

Unmasked positions contribute *zero* gradient from the output head, because they are not in the sum. That is why the masking rate controls how much learning signal each sequence gives.

Beyond the slides : weight tying

In the original BERT implementation, the output matrix W_v of the MLM head is **tied** to (is the same matrix as) the token-embedding matrix $E_T \in \mathbb{R}^{|\mathcal{V}| \times d}$, and there is a small transform layer (dense + GELU + LayerNorm) before it. Tying saves $|\mathcal{V}| \cdot d \approx 23\text{M}$ parameters and makes the logit for word w equal to $\langle e_w, h_i \rangle$. In words: “how similar is the context vector to word w ’s embedding?”

How much to mask? The 15% rule

Typically **15% of the tokens** in the input sequence are masked.

- **Too much masking** destroys the context. If 80% of the words are [MASK], there is not enough left to infer the blanks, and the model does not learn good representations.
- **Too little masking** makes training slow. Only $|\mathcal{M}|$ positions produce gradients, so with 1% masking each sequence contributes very few prediction targets and the total gradient signal per step is small. Convergence takes a long time, so training is inefficient.
- The right trade-off depends on model size, masking scheme and optimiser. (Wettig et al., 2022, “Should You Mask 15% in Masked Language Modeling?” found that larger models can profit from masking as much as 40%.)

Intuition : the context-vs-signal trade-off

Let r be the masking rate and T the sequence length. The number of training targets per sequence is rT , which grows with r . The average amount of context available to each target is roughly $(1 - r)T$, which shrinks with r . The quality of learning needs both, so there is a sweet spot. For BERT-base with $T = 512$, $r = 0.15$ gives about 77 targets per sequence. GPT gets $T = 512$ targets per sequence, one per position, which is one reason MLM is often called less *sample-efficient* than causal LM.

2.6 BERT: the architecture

BERT is a multi-layer bidirectional transformer encoder. BERT-base has **12 layers** with **12 attention heads per layer**. Each encoder layer contains multi-head self-attention, a feed-forward network, layer normalisation and residual connections.

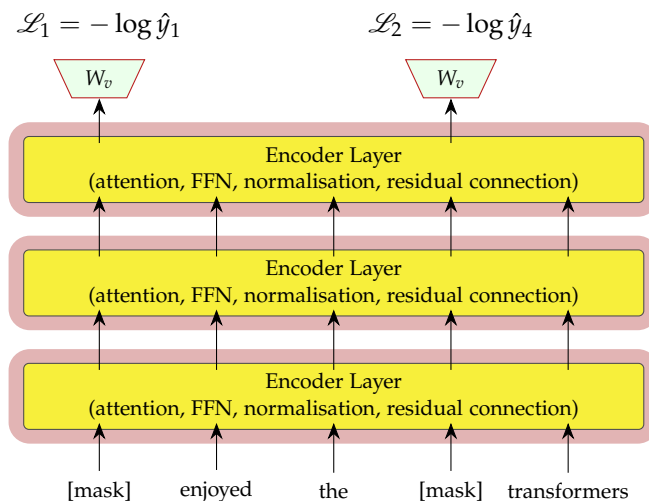


Figure 8: BERT stacks L encoder layers. The MLM head sits on top of the last layer at the masked positions.

The 80/10/10 rule

The 15% of positions chosen for prediction are **sampled uniformly**. Of these:

- **80%** are replaced by [MASK]: “my dog is hairy” → “my dog is [MASK]”

- 10% are replaced by a **random word**: “my dog is hairy” → “my dog is apple”
- 10% are **left unchanged**: “my dog is hairy” → “my dog is hairy”

In *all three* cases, the model must predict the original word (“hairy”) at that position, and that position contributes to the loss.

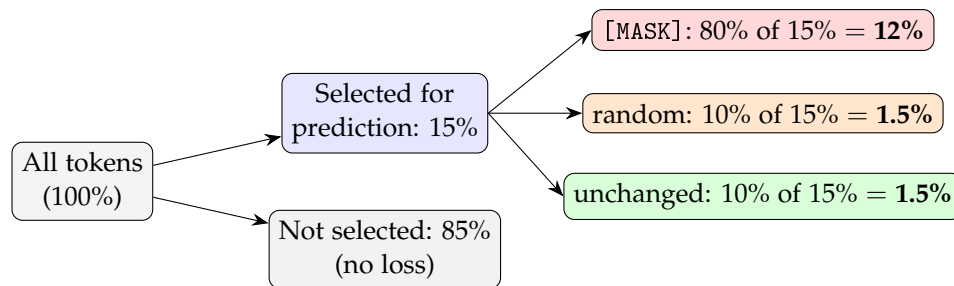


Figure 9: The masking tree. Percentages on the right are fractions of *all* tokens.

Why? (the slide asks this).

1. **Pre-train / fine-tune mismatch.** The special [MASK] token *never appears* in downstream data (sentiment reviews, questions, and so on). If the model only ever saw [MASK] at prediction positions, it could learn “only build a useful contextual representation where I see [MASK]”. At fine-tuning time there are no [MASK] tokens, so the representations would be off-distribution. Keeping some chosen tokens as real words (random or unchanged) forces the model to build good contextual representations for *every* token.
2. **Random replacement (10%)** teaches the model that *the observed word might be wrong*. It cannot blindly trust or copy its input, so it must check every token against its context. This acts as a regulariser and helps robustness.
3. **Unchanged (10%)** biases the representation towards the actually observed word. Without it, the model would learn that “whenever a position is being predicted, the visible word is wrong”. Because the model does not know *which* positions are being predicted, it has to keep a representation that is faithful to the input *and* informed by context.

Common pitfall / exam trap

Random replacement happens to only 1.5% of all tokens, so it barely hurts the model’s language understanding. Students often think “10% random words is a lot of noise”. It is 10% of the 15%.

Systems / engineering note : dynamic vs static masking

Original BERT chose the masks once during data preprocessing (*static masking*) and duplicated the data 10 times with different masks. RoBERTa (2019) samples a new mask every time a sequence is fed to the model (*dynamic masking*), which is simpler and slightly better. Most modern MLM data collators (for example HuggingFace’s `DataCollatorForLanguageModeling`) mask dynamically on the fly.

Is MLM enough? The slide asks: *is MLM alone sufficient for downstream tasks like Question Answering, where the interaction between sentences is important?* MLM trains the model to understand words in context within a stretch of text. It never explicitly asks “how do two separate pieces of text relate?” That question motivates the second pre-training objective.

2.7 Next Sentence Prediction (NSP)

Extend the input to a **pair of sentences** (A, B) , together with a label saying whether B naturally follows A .

input: Sentence A: *I enjoyed the movie transformers*
input: Sentence B: *The visuals were amazing*
Label: IsNext

- The two sentences are separated by a special token [SEP]. (A [SEP] also ends the pair.)
- A special token [CLS] is placed at the very beginning. Its **final hidden representation is used for the classification** (IsNext / NotNext).
- **50%** of the time, B is the actual next sentence (IsNext). **50%** of the time, B is a random sentence from the corpus (NotNext).
- Pre-training with NSP was reported to **significantly improve QA and similar tasks** (natural language inference and so on).

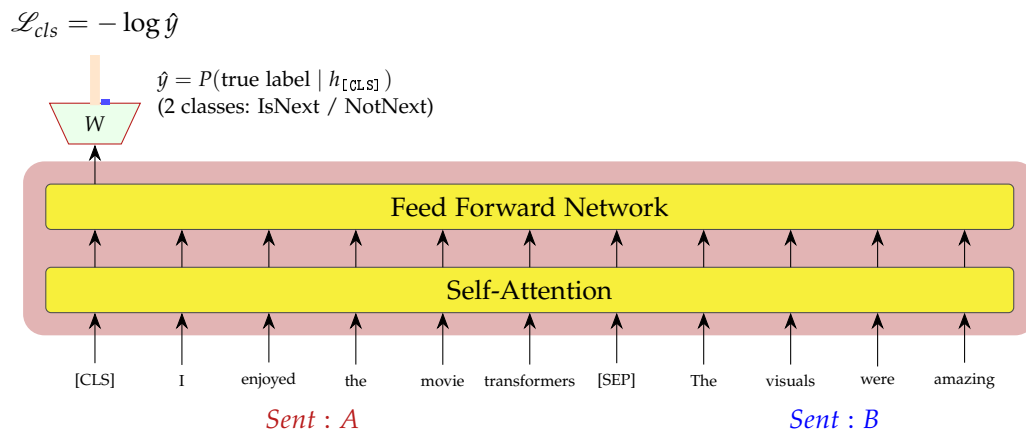


Figure 10: NSP: the [CLS] output feeds a 2-way classifier.

Intuition : why [CLS], and why does it work?

[CLS] is a token with *no* meaning of its own. Because every layer's self-attention lets it attend to all tokens of both sentences, and because the NSP loss is applied *only* at its output, training pushes its final hidden state to become a **summary of the whole pair** that is useful for deciding "do these go together?". Any real word position would be a worse choice, since its representation is also busy describing that particular word.

Segment embeddings: which sentence does a token belong to?

The model needs to know which tokens belong to A and which to B . [SEP] alone is a weak signal, because a token far from [SEP] would have to work it out through attention. BERT therefore adds a separate **learnable segment embedding** (E_A or E_B), in addition to the token and position embeddings:

Formula : BERT input representation

For the token at position t with vocabulary id x_t and segment $s_t \in \{A, B\}$:

$$\mathbf{e}_t = \underbrace{E_T[x_t]}_{\text{token}} + \underbrace{E_S[s_t]}_{\text{segment}} + \underbrace{E_P[t]}_{\text{position}} \quad \mathbf{e}_t \in \mathbb{R}^d,$$

with $E_T \in \mathbb{R}^{|\mathcal{V}| \times d}$, $E_S \in \mathbb{R}^{2 \times d}$ and $E_P \in \mathbb{R}^{T_{\max} \times d}$ (**learned**, not sinusoidal, in BERT). The three are *added*, not concatenated, so the model width stays d . BERT then applies LayerNorm and dropout to $\mathbf{e}_t$.

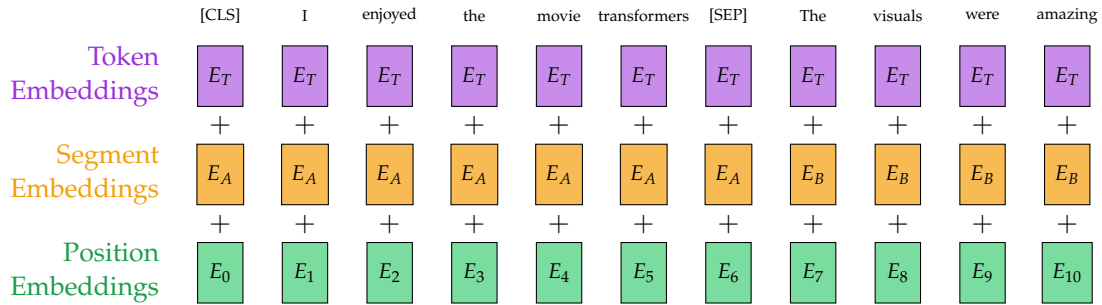


Figure 11: Input embedding = token + segment + position. Note that [SEP] (the separator after sentence A) gets segment E_A .

Common pitfall / exam trap : adding vs concatenating

Students often ask “doesn’t adding the three vectors mix them up irrecoverably?” In a high-dimensional space ($d = 768$), the network can learn embeddings that occupy roughly separate subspaces. Adding costs nothing extra and keeps the width fixed. Concatenating would make the input $3d$ wide and need an extra projection.

2.8 Putting MLM and NSP together

During pre-training, the input is a masked sentence pair, and both losses are computed at the same time:

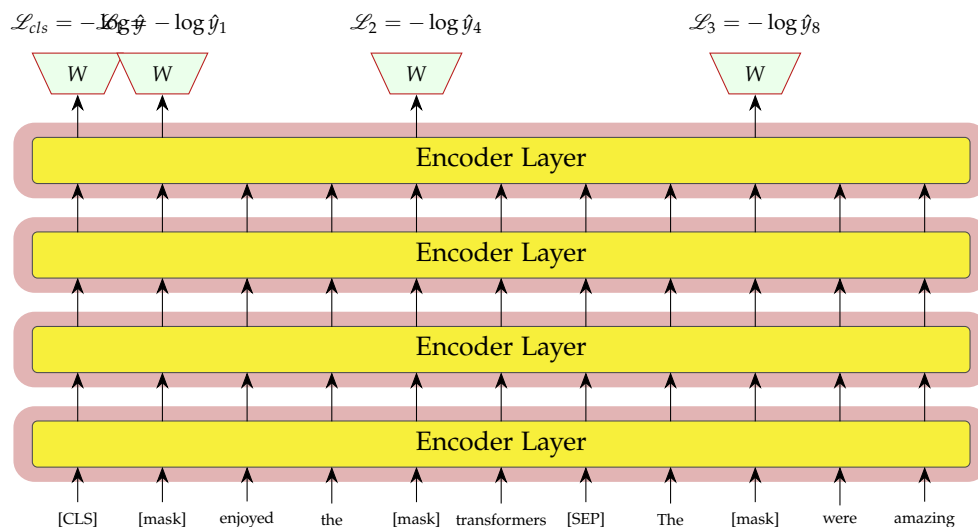


Figure 12: Joint pre-training: NSP loss at [CLS], MLM loss at every masked position.

Formula : total BERT pre-training objective

$$\mathcal{L} = \underbrace{\frac{1}{|\mathcal{M}|} \sum_{y_i \in \mathcal{M}} -\log(\hat{y}_i)}_{\text{MLM}} + \underbrace{\mathcal{L}_{cls}}_{\text{NSP}}, \quad \mathcal{L}_{cls} = -\log P(\text{label} \mid h_{[\text{CLS}]}).$$

The two terms are simply added with equal weight. In practice each is averaged over the batch.

Beyond the slides : what happened to NSP later

RoBERTa (Liu et al., 2019) found that **removing NSP** and training on contiguous full-length text matched or improved downstream results. The explanation: a random NotNext sentence usually comes from a different *topic*, so NSP can mostly be solved by topic detection, which MLM already learns. ALBERT replaced NSP with **Sentence Order Prediction (SOP)**: *B* is either the true next sentence or the *same two sentences swapped*, which forces the model to learn coherence rather than topic. For exams, stick to the lecture: BERT uses MLM + NSP.

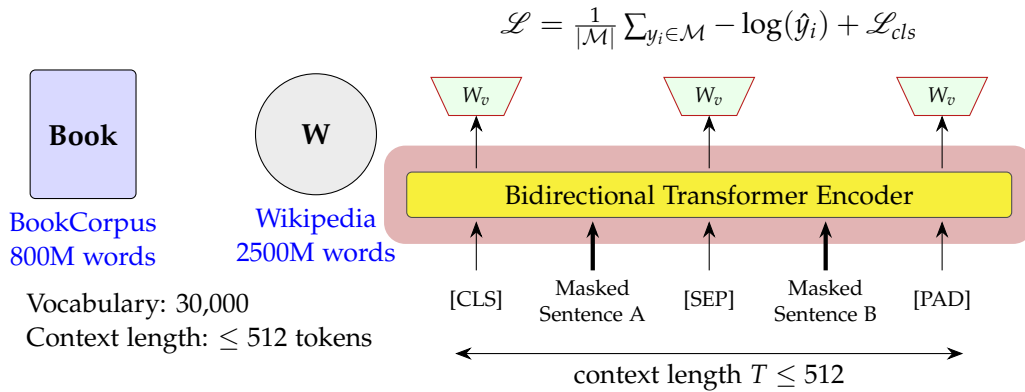
2.9 Pre-training setup

Figure 13: Pre-training data and the input format: [CLS] + masked A + [SEP] + masked B (+ [SEP]) + padding up to T .

Hyper-parameter	BERT-base	BERT-large
Pre-training data	BookCorpus (800M words) + English Wikipedia (2,500M words)	
Vocabulary $ \mathcal{V} $	$\approx 30,000$ WordPiece tokens (exactly 30,522)	
Max context length T	512	512
Number of layers L	12	24
Hidden size $H = d_{\text{model}}$	768	1024
Attention heads A	12	16
Head dimension $d_k = H/A$	64	64
FFN intermediate size $4H$	3072	4096
Total parameters	$\approx 110\text{M}$	$\approx 340\text{M}$

The hidden state at every position and at every layer boundary has size $h \in \mathbb{R}^{768}$ (for base). The residual stream has a constant width through the whole stack.

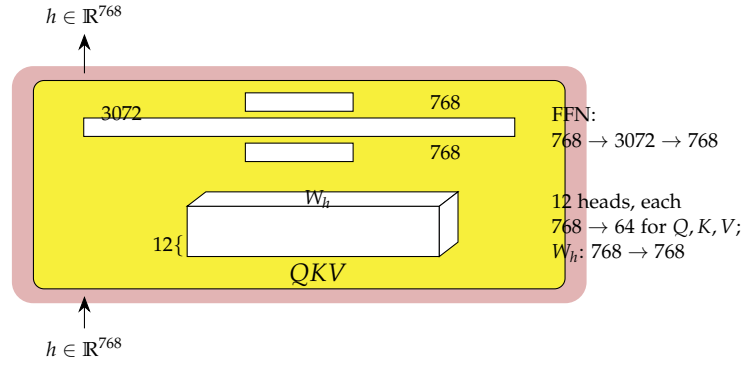


Figure 14: Inside one BERT-base encoder layer: 12 heads of Q, K, V projections, an output projection W_h , and a $768 \rightarrow 3072 \rightarrow 768$ feed-forward network.

Beyond the slides : training recipe from the BERT paper

Batch of 256 sequences $\times$ 512 tokens ($\approx 128K$ tokens per batch), 1,000,000 steps (≈ 40 epochs over the 3.3B-word corpus), Adam with learning rate 10^{-4} , warm-up of 10K steps, dropout 0.1, GELU activations. To save compute, 90% of the steps used sequence length 128, and only the last 10% used 512 (which is how the longer position embeddings get trained). The two pre-training tasks share the same forward pass.

2.10 Parameter calculation (BERT-base)

This is a favourite exam topic, so we go through it slowly.

Embedding layer

$$\begin{aligned} \text{Token embeddings } E_T : |\mathcal{V}| \times d &= 30,000 \times 768 = 23,040,000 \approx 23M \\ \text{Segment embeddings } E_S : 2 \times 768 &= 1,536 \\ \text{Position embeddings } E_P : 512 \times 768 &= 393,216 \approx 0.4M \\ \text{Total embeddings} &\approx 23.4M \end{aligned}$$

One encoder layer (ignoring biases, as the slide does)

Self-attention. Each of the 12 heads has $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)} \in \mathbb{R}^{768 \times 64}$:

$$(768 \times 64 \times 3) \times 12 = 147,456 \times 12 = 1,769,472 \approx 1.7M.$$

Note that $64 \times 12 = 768$, so this is the same as three 768×768 matrices: $3 \times 768^2 = 1,769,472$. The **output projection** A (called W_O or W_h) maps the concatenated heads (768) back to 768:

$$768 \times 768 = 589,824 \approx 0.6M.$$

Feed-forward network $\text{FFN}(h) = W_2 \text{GELU}(W_1 h + b_1) + b_2$ with $W_1 \in \mathbb{R}^{3072 \times 768}$ and $W_2 \in \mathbb{R}^{768 \times 3072}$. The slide includes the FFN biases:

$$768 \times 3072 + 3072 \times 768 + (3072 + 768) = 2,359,296 + 2,359,296 + 3,840 = 4,722,432 \approx 4.7M.$$

Per-layer total: $1.77 + 0.59 + 4.72 \approx 7.08M \approx 7M$.

All 12 layers and grand total

12 layers : $12 \times 7\text{M} \approx 84\text{M}$ (more precisely $12 \times 7.08\text{M} = 84.98\text{M}$)

Total $\approx 23.4 + 84 = \mathbf{107.4\text{M}}$ (110M in the paper)

The slide's footnote: the actual vocabulary size is 30,522, and layer-normalisation parameters (and some biases) are excluded. Here is the exact count including everything, which is a useful cross-check:

Component (BERT-base, exact)	Parameters
Token emb. 30522×768	23,440,896
Position emb. 512×768	393,216
Segment emb. 2×768	1,536
Embedding LayerNorm $(\gamma, \beta) 2 \times 768$	1,536
<i>Embeddings subtotal</i>	<i>23,837,184</i>
Per layer: Q, K, V weights $3 \times 768^2 + \text{biases } 3 \times 768$	1,771,776
Per layer: output proj. $768^2 + 768$	590,592
Per layer: LayerNorm after attention	1,536
Per layer: FFN $W_1 (768 \cdot 3072 + 3072)$	2,362,368
Per layer: FFN $W_2 (3072 \cdot 768 + 768)$	2,360,064
Per layer: LayerNorm after FFN	1,536
<i>Per layer subtotal</i>	<i>7,087,872</i>
<i>12 layers</i>	<i>85,054,464</i>
Pooler (dense on [CLS]: $768^2 + 768$)	590,592
Total	109,482,240 $\approx$ 110M

Formula : general closed form (per layer, ignoring biases and LN)

With hidden size d and FFN size $4d$:

$$\underbrace{4d^2}_{Q,K,V,O} + \underbrace{8d^2}_{\text{FFN}} = 12d^2 \Rightarrow N_{\text{layers}} \approx 12 L d^2, \quad N_{\text{emb}} = (|\mathcal{V}| + T_{\text{max}} + 2) d.$$

BERT-base: $12 \cdot 12 \cdot 768^2 = 84.9\text{M}$. BERT-large: $12 \cdot 24 \cdot 1024^2 = 302\text{M}$, plus embeddings $(30522 + 512 + 2) \cdot 1024 = 31.8\text{M}$, for a total of $\approx 334\text{M}$ ($\approx 340\text{M}$ in the paper once biases, LN and the pooler are included).

Intuition : where do the parameters live?

In BERT-base, about $\frac{23.4}{110} \approx 21\%$ of the parameters are the embedding table, and about 2/3 of each layer's parameters are in the FFN ($8d^2$ vs $4d^2$). The number of heads does *not* change the parameter count, because $A \times (d \times d / A) = d^2$. Heads split the same capacity into parallel sub-spaces.

Systems / engineering note : compute and memory, back of the envelope

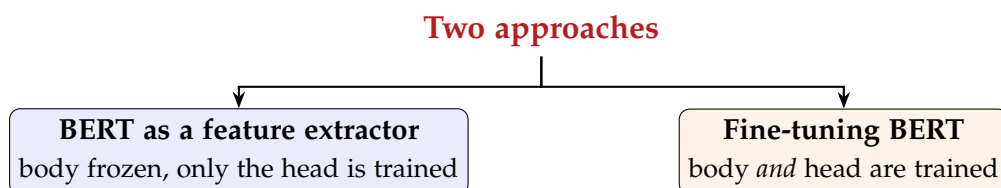
- **Memory for weights:** $110M \times 4$ bytes (fp32) $\approx 440MB$. In fp16/bf16 it is $\approx 220MB$. Training with Adam needs roughly 16 bytes/param (weights + grads + 2 moments + fp32 master copy in mixed precision), so $\approx 1.8GB$ before activations.
- **FLOPs:** a forward pass costs $\approx 2N$ FLOPs per token (one multiply-add per weight), and training costs $\approx 6N$ FLOPs per token (forward + backward $\approx 3 \times$ forward). Excluding embeddings (lookups are nearly free), $N \approx 85M$, so ≈ 0.17 GFLOP/token forward.
- **Attention cost:** the $QK^\top$ and AV matmuls cost $\approx 4T^2d$ FLOPs per layer. They grow *quadratically* with T , which is one reason BERT capped T at 512 and trained mostly at $T = 128$.

Summary of Module 3.1

- BERT = stack of transformer *encoders*; bidirectional attention with no causal mask.
- We cannot hide words by masking attention (the residual stream leaks them, and the blank should be a real input), so we **corrupt the input** with a learnable [MASK] token. MLM is a denoising objective.
- Loss: average $-\log \hat{y}_i$ over masked positions only. Predictions are treated as independent.
- 15% of tokens are chosen: 80% $\rightarrow$ [MASK], 10% $\rightarrow$ random, 10% unchanged, to bridge the pre-training/fine-tuning gap.
- NSP: [CLS] A [SEP] B [SEP], 50/50 IsNext/NotNext, classifier on $h_{[CLS]}$. Segment embeddings mark A vs B.
- Input = token + segment + position embeddings (all learned, summed).
- BERT-base: $L = 12, H = 768, A = 12$, FFN 3072, $|\mathcal{V}| \approx 30K$, $T \leq 512$, $\approx 110M$ params.

3 Module 3.2: Adapting BERT to downstream tasks

Pre-training gives us a network that turns any token sequence into rich contextual vectors. To use it for a real task (sentiment classification, NLI, QA, NER, and so on) there are **two approaches**:



3.1 Classification, feature-based approach

1. Take a sentence (or pair) of length < 512 and feed it to BERT, **padding** if needed.
2. Take the **final hidden representation** (output of the last encoder) as a **feature vector for the entire sentence**. Usually this is $h_{[CLS]}$, or the mean of all token vectors.
3. Train any ML model on top of it (the **head**), such as logistic regression, Naive Bayes or a small neural network, to predict $\hat{y} \in \{0, 1\}$.
4. **All parameters of BERT are frozen**. Only the classification head is trained, from scratch.

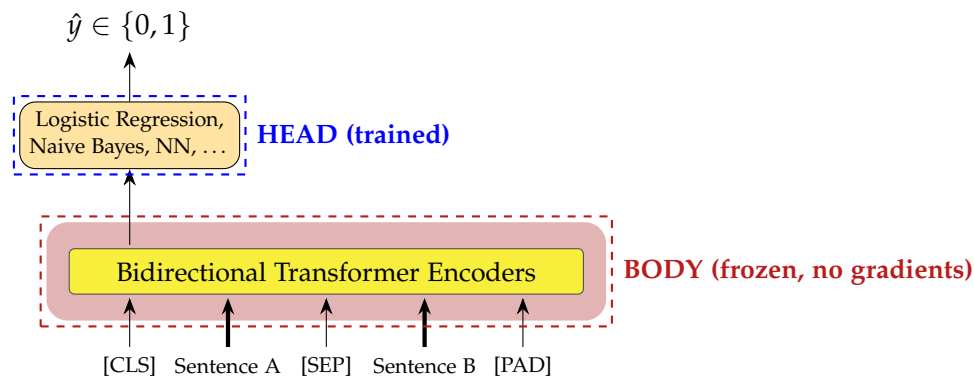


Figure 15: Feature-based use: BERT is a fixed function $x \mapsto h$, and only the head learns.

Key idea : body and head

It is useful to split any adapted model into a **body** (the pre-trained encoder that produces representations) and a **head** (a small task-specific model that maps representations to outputs). The two approaches differ only in *whether the body is also updated*.

Why is this better than concatenating word2vec vectors? The slides say the BERT representation is **superior to merely concatenating representations of individual words** (say from word2vec).

- word2vec gives each word *one static vector*, so “bank” in “river bank” and in “bank loan” gets the same vector. BERT vectors are *contextual*: each is computed from the whole sentence.
- Concatenation gives a vector whose length depends on the sentence length, and word order and interactions are left for the head to figure out. BERT’s $h_{[\text{CLS}]}$ is a fixed-size vector that has already mixed all the words through 12 layers of attention.
- Negation, for instance: “not good” vs “good”. Composing meaning like this is done *by BERT’s layers*, not left to a weak head such as logistic regression.

Systems / engineering note : why anyone still uses feature extraction

Frozen features can be **pre-computed once and cached**. You run BERT over the dataset once, store the 768-d vectors, and then train many cheap heads (grid search, different tasks) without ever running BERT’s backward pass. That is also how **embedding-based retrieval / RAG** systems work: a frozen (or separately fine-tuned) encoder turns documents into vectors stored in a vector DB such as FAISS or Qdrant.

3.2 Classification, fine-tuning approach

1. Add a classification head (again, any suitable ML model; typically one linear layer + softmax on $h_{[\text{CLS}]}$).
2. **Initialise the head’s parameters randomly.**
3. **Train the entire model**, including all pre-trained BERT parameters, on the new labelled dataset.
4. **We do not mask words in the input sequence** during fine-tuning. This is exactly why 10% of the chosen words were replaced by random words (and 10% kept) during pre-training: so that the model learned to produce useful representations for *unmasked* real tokens.

5. It is observed that the model **converges quickly** and needs **fewer labelled training samples** than the feature-based approach.

Formula : a standard fine-tuning head for C -class classification

$$\hat{\mathbf{p}} = \text{softmax}(W_c h_{[\text{CLS}]} + b_c), \quad W_c \in \mathbb{R}^{C \times 768}, \quad b_c \in \mathbb{R}^C, \quad \mathcal{L} = -\log \hat{p}_y.$$

For sentiment ($C = 2$) the head has only $2 \times 768 + 2 = 1,538$ new parameters. Gradients of $\mathcal{L}$ flow through $h_{[\text{CLS}]}$ into *all* 110M BERT parameters.

Intuition : why does fine-tuning need fewer labels?

In the feature-based approach, the representation is fixed: if the information the task needs is not linearly readable from $h_{[\text{CLS}]}$, a simple head cannot recover it. In fine-tuning, the body *reshapes* its representation a little towards the task, starting from an excellent initialisation. The model begins very close to a good solution, so a few thousand labelled examples and 2–4 epochs are typically enough. Training from scratch would need orders of magnitude more data.

	Feature-based	Fine-tuning
BERT parameters	Frozen	Updated
Head	Any ML model, trained from scratch	Usually linear/softmax, random init
Trainable params	Only head (e.g. $\sim 1.5\text{K}$)	All ($\sim 110\text{M} + \text{head}$)
Labelled data needed	More	Less (converges quickly)
Compute per task	Low (can cache features)	High (full backward pass)
Storage per task	One tiny head	A full 110M copy (unless PEFT is used)
Task fit	Limited by fixed features	Representation adapts to task

Beyond the slides : the middle ground

Between “freeze everything” and “train everything” are **parameter-efficient fine-tuning** (PEFT) methods: adapters, prefix tuning, and **LoRA** (freeze W , learn a low-rank update $\Delta W = BA$ with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times d}$, $r \ll d$). These keep most of fine-tuning’s quality while storing only a small set of new weights per task. Also common is *partial* freezing, for example training only the top k layers.

3.3 Extractive Question Answering

In extractive QA, the answer is a **contiguous span** of a given paragraph. The model does not generate new text. It points to where the answer starts and ends.

Pre-training uses unlabelled data (BookCorpus + Wikipedia) with MLM + NSP, as before.

Fine-tuning uses labelled data (such as SQuAD): triples of question, paragraph and answer span.

Worked example : from the slides

Question: What is unique about the mission?

Paragraph: What sets this mission apart is the pivotal role of artificial intelligence (AI) in guiding the spacecraft during its critical descent to the moon's surface.

Answer: role of artificial intelligence (AI) in guiding the spacecraft

Starting token: 9 ("role"). **Ending token:** 17 ("spacecraft"), counting paragraph words from 1:

What ₁	sets ₂	this ₃	mission ₄	apart ₅	is ₆	the ₇	pivotal ₈	role ₉
of ₁₀	artificial ₁₁	intelligence ₁₂	(AI) ₁₃	in ₁₄	guiding ₁₅	the ₁₆	spacecraft ₁₇	... ₂₅

Input format. [CLS] *question tokens* [SEP] *paragraph tokens* [SEP]. The question goes in segment A, the paragraph in segment B. The encoder produces final representations $h_1, h_2, \dots, h_{25}$ for the 25 paragraph tokens. We use them to find the **start** and **end** tokens.

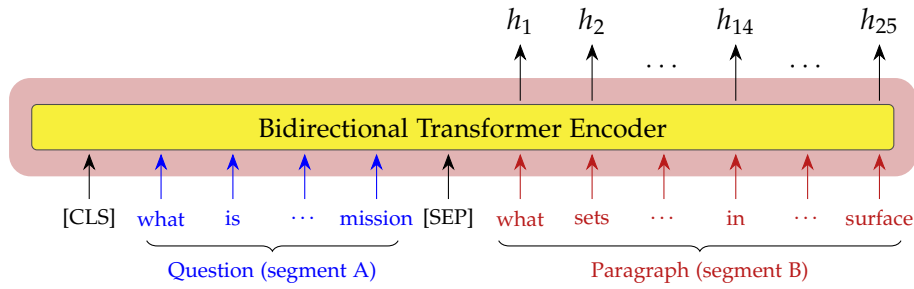


Figure 16: The question and paragraph are packed into one input. We read out a vector per paragraph token.

Start and end vectors

Formula : span-prediction head

Let $S \in \mathbb{R}^d$ be a learnable **start vector** and $E \in \mathbb{R}^d$ a learnable **end vector**, both the same size as h_i ($d = 768$). The probability that the i -th paragraph word is the start token is

$$s_i = \frac{\exp(S \cdot h_i)}{\sum_{j=1}^{25} \exp(S \cdot h_j)},$$

and the probability that it is the end token is

$$e_i = \frac{\exp(E \cdot h_i)}{\sum_{j=1}^{25} \exp(E \cdot h_j)}.$$

Both S and E are learnable parameters, so the QA head adds only $2d = 1,536$ parameters.

Dimensions. $S \cdot h_i$ is a dot product of two $\mathbb{R}^{768}$ vectors, so it is a scalar score. Stack the paragraph states as $H_p \in \mathbb{R}^{25 \times 768}$. Then $H_p S \in \mathbb{R}^{25}$ gives all start logits in one matrix–vector product, and $\mathbf{s} = \text{softmax}(H_p S) \in \mathbb{R}^{25}$ is a probability distribution over paragraph positions. The same holds for $\mathbf{e}$. Note that the softmax runs over **positions**, not over the vocabulary.

Intuition : what S and E learn

S is a “prototype of an answer-start” direction in representation space. Fine-tuning pushes the hidden state of the true start token (whose representation, thanks to bidirectional attention, already “knows” the question) to align with S . It pushes the others away. It is a learned *pointer*.

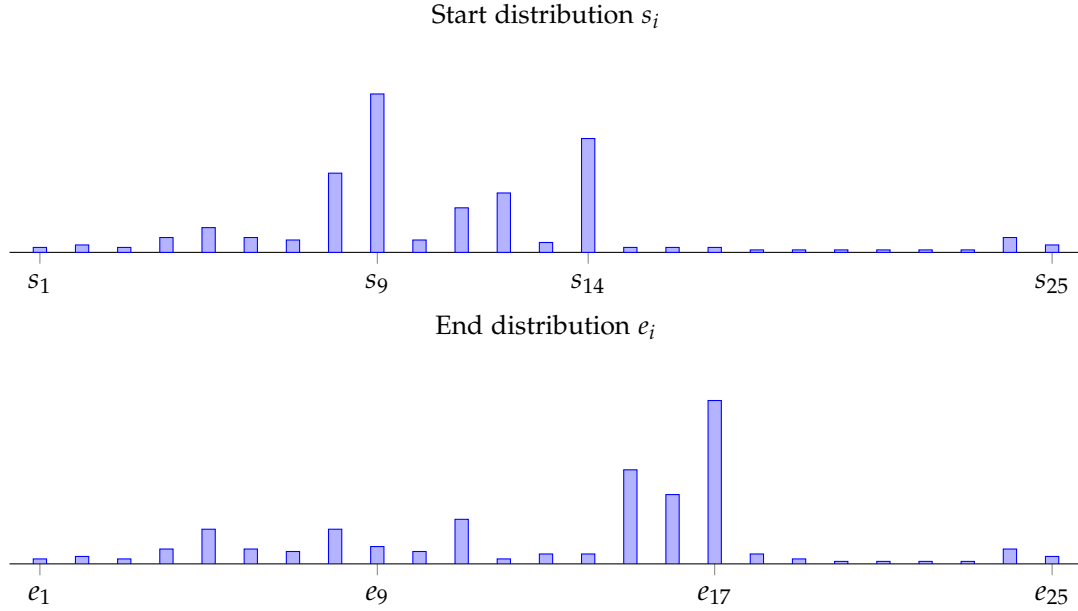


Figure 17: Illustrative start and end distributions over the 25 paragraph positions. The start peaks at 9 (“role”), the end at 17 (“spacecraft”).

Decoding the span

Choose the start as the most probable start position i and the end as the most probable end position j . The answer is the span of words $i, \dots, j$. It is valid if $j \geq i$.

- In the example, $i = 9$ and $j = 17$, so the span is “role of artificial intelligence (AI) in guiding the spacecraft”.
- **It is possible that the end index is smaller than the start index**, for example e_7 (“the”) is the highest end probability while s_9 is the highest start probability. Then $j = 7 < i = 9$, and we **return an empty string**. This implies that the answer is not in the paragraph.

Beyond the slides : how it is trained and decoded in practice

Training loss. With gold start i^* and gold end j^* ,

$$\mathcal{L}_{QA} = -\frac{1}{2}(\log s_{i^*} + \log e_{j^*}).$$

Two independent cross-entropies over positions. Gradients flow into S , E and all of BERT.

Better decoding. Instead of taking the two argmaxes independently, the BERT paper picks the pair

$$(\hat{i}, \hat{j}) = \arg \max_{i \leq j \leq i + L_{\max}} (S \cdot h_i + E \cdot h_j),$$

the best *valid* span, with a maximum length $L_{\max}$ (e.g. 30 tokens). This is $O(nL_{\max})$ instead of $O(n^2)$. For SQuAD 2.0 (unanswerable questions), the “no answer” score is $S \cdot h_{[\text{CLS}]} + E \cdot h_{[\text{CLS}]}$. The model predicts “no answer” when that beats the best non-null span by a tuned threshold. This is a cleaner version of the slides’ “return empty string when $j < i$ ”.

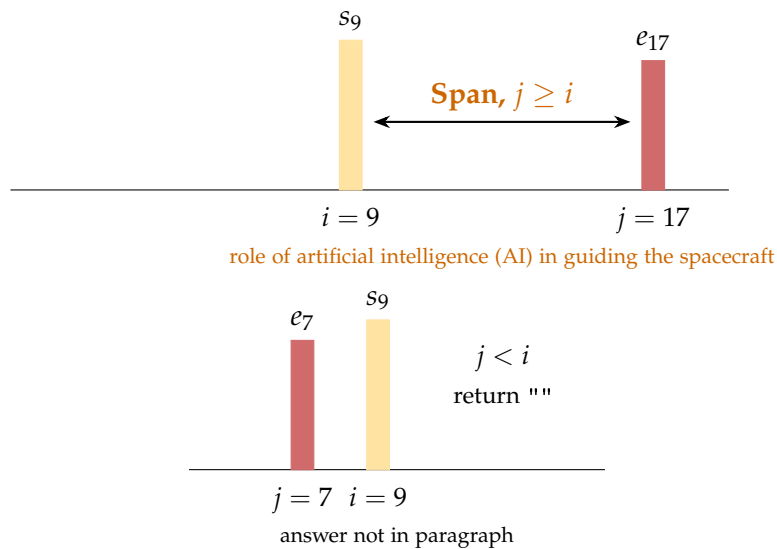


Figure 18: Top: a valid span. Bottom: an invalid span ($j < i$), so we output the empty string.

Common pitfall / exam trap : tokens vs words

The slides count *words*. Real BERT operates on **WordPiece sub-tokens** (next week!), so “spacecraft” may be split into several pieces, and the start/end indices refer to sub-token positions, which are then mapped back to character offsets in the paragraph. Paragraphs longer than 512 tokens are split into overlapping windows (“doc stride”).

3.4 Other downstream tasks (brief)

Task	Input	Head / output
Single-sentence classification (sentiment)	[CLS] sent [SEP]	Linear + softmax on $h_{[\text{CLS}]}$
Sentence-pair classification (NLI, paraphrase)	[CLS] A [SEP] B [SEP]	Linear + softmax on $h_{[\text{CLS}]}$
Token tagging (NER, POS)	[CLS] sent [SEP]	Linear + softmax on <i>every</i> h_t (per-token label)
Extractive QA	[CLS] Q [SEP] P [SEP]	Start/end vectors S, E over paragraph h_t

This connects back to the NER example in [Section 1](#): with BERT, the tag of the first “Nothing” comes from h_1 , which has attended to “Phone 2” at the end of the sentence.

Summary of Module 3.2

- Two approaches: **feature extraction** (freeze BERT, train only a head) and **fine-tuning** (train everything; head randomly initialised).
- Fine-tuning uses **no masking**. It converges fast and needs fewer labels. That is the payoff of the 80/10/10 rule.
- Extractive QA: learn $S, E \in \mathbb{R}^{768}$, $s_i = \text{softmax}_i(S \cdot h_i)$ and $e_i = \text{softmax}_i(E \cdot h_i)$. The

answer span is $[i, j]$ with $j \geq i$. If $j < i$, return an empty string (no answer).

4 Quick-revision cheat sheet

Concept	Must-remember fact
BERT	Encoder-only, bidirectional, pre-trained with MLM + NSP, fine-tuned for understanding tasks.
MLM objective	$P(y_i x_1, \dots, x_i = [\text{MASK}], \dots, x_T)$; masked tokens predicted independently.
Why not attention mask	The blank must be an input the model attends from; the residual would leak the true token. So corrupt the input (denoising).
MLM loss	$\mathcal{L} = \frac{1}{ \mathcal{M} } \sum_{i \in \mathcal{M}} -\log \hat{y}_i$, only over masked positions.
Masking rate	15% of tokens (too high loses context; too low gives slow convergence).
80/10/10	Of the 15%: 80% $\rightarrow$ [MASK] (12% overall), 10% random (1.5%), 10% same (1.5%). Reason: [MASK] is absent at fine-tuning.
NSP	[CLS] A [SEP] B [SEP]; 50% IsNext, 50% NotNext (random); classifier on $h_{[\text{CLS}]}$.
Embeddings	token + segment + position (learned), summed, each $\in \mathbb{R}^{768}$.
Total loss	$\mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{cls}}$.
Data	BookCorpus 800M + Wikipedia 2500M words; $ \mathcal{V} \approx 30\text{K}$; $T \leq 512$.
Base / Large	$L=12/24$, $H=768/1024$, $A=12/16$, $\text{FFN}=3072/4096$, $\approx 110\text{M}/340\text{M}$.
Params (slide)	Emb. 23.4M; per layer $1.7 + 0.6 + 4.7 \approx 7\text{M}$; $\times 12 = 84\text{M}$; total $\approx 107.4\text{M}$ (110M in paper).
Adaptation	Feature-based (freeze body, train head) vs fine-tuning (train all, no masking, converges fast).
Extractive QA	$s_i = \text{softmax}_i(S \cdot h_i)$, $e_i = \text{softmax}_i(E \cdot h_i)$; $S, E \in \mathbb{R}^{768}$ learnable; span $[i, j]$, $j \geq i$; if $j < i$ return empty string.

5 Numerical problems with worked solutions

Problem 1. Masking arithmetic

A pre-training sequence has 512 tokens, of which 3 are special ([CLS], [SEP], [SEP]) and never masked. BERT's rule is applied to the remaining tokens. (a) How many tokens are selected for prediction (expected)? (b) How many of them become [MASK], random, and unchanged? (c) What is the probability that a given non-special token appears as [MASK] in the input? (d) As a random token?

Solution

Maskable tokens: $512 - 3 = 509$. (a) $0.15 \times 509 = 76.35 \approx 76$ tokens selected. These are $|\mathcal{M}|$, the positions that enter the loss. (b) [MASK]: $0.8 \times 76.35 = 61.08 \approx 61$. Random: $0.1 \times 76.35 = 7.6$. Unchanged: 7.6. In a real sequence that is about 8 and 7 (or 7 and 8), so the three counts add up to 76. (c) $P([\text{MASK}]) = 0.15 \times 0.8 = 0.12$. (d) $P(\text{random}) = 0.15 \times 0.1 = 0.015$. Note that the loss is computed on all ≈ 76 selected positions, *including* the ≈ 8 unchanged ones.

Problem 2. *MLM loss*

In one sequence, 3 positions are masked. The model assigns probabilities 0.6, 0.2 and 0.05 to the correct tokens at these positions. Compute $\mathcal{L}_{\text{MLM}}$ (natural log). If the NSP classifier gives probability 0.9 to the correct label, compute the total loss.

Solution

$$-\ln 0.6 = 0.5108, \quad -\ln 0.2 = 1.6094, \quad -\ln 0.05 = 2.9957.$$

$$\mathcal{L}_{\text{MLM}} = \frac{1}{3}(0.5108 + 1.6094 + 2.9957) = \frac{5.1160}{3} = \mathbf{1.7053}.$$

$\mathcal{L}_{\text{cls}} = -\ln 0.9 = 0.1054$. Total $\mathcal{L} = 1.7053 + 0.1054 = \mathbf{1.8107}$. The hardest position ($p = 0.05$) contributes over half the MLM loss. Cross-entropy punishes confident mistakes heavily.

Problem 3. *Softmax + cross-entropy gradient*

At a masked position, the MLM head produces logits $z = (2.0, 1.0, 0.1)$ over a toy vocabulary {a, b, c}. The true token is *b*. Find $\hat{\mathbf{p}}$, the loss, and $\partial \mathcal{L} / \partial z$.

Solution

$e^2 = 7.389$, $e^1 = 2.718$, $e^{0.1} = 1.105$, sum = 11.212. So $\hat{\mathbf{p}} = (0.659, 0.242, 0.099)$. Loss = $-\ln 0.242 = \mathbf{1.417}$. Gradient: $\hat{\mathbf{p}} - \mathbf{t} = (0.659, 0.242 - 1, 0.099) = (\mathbf{0.659}, \mathbf{-0.758}, \mathbf{0.099})$. Gradient descent therefore *raises* z_b and lowers z_a, z_c , each in proportion to how wrong it currently is.

Problem 4. *Attention with the slide's [MASK] vectors*

From the slides: for the [MASK] token (embedding $[1, 0, 0]$), $q_1 = [-0.1, 0.25]$, $k_1 = [0.3, 0.2]$, $v_1 = [0.1, 0.5]$. For “transformers” ($[1, 1, 1]$), $q_5 = [0.2, 0.7]$, $k_5 = [0.11, 0.89]$, $v_5 = [0, 0.4]$. Consider only these two tokens with $d_k = 2$ and no mask. Compute the attention output z_1 for the [MASK] position.

Solution

Scores: $q_1 \cdot k_1 = (-0.1)(0.3) + (0.25)(0.2) = -0.03 + 0.05 = 0.02$. $q_1 \cdot k_5 = (-0.1)(0.11) + (0.25)(0.89) = -0.011 + 0.2225 = 0.2115$.

Scale by $1/\sqrt{2}$: 0.01414 and 0.14955.

Softmax: the difference is 0.13541, so $\alpha_{15} = \frac{1}{1+e^{-0.13541}} = 0.5338$ and $\alpha_{11} = 0.4662$.

Output: $z_1 = 0.4662 [0.1, 0.5] + 0.5338 [0, 0.4] = [0.0466, 0.2331 + 0.2135] = [\mathbf{0.0466}, \mathbf{0.4466}]$.

The [MASK] position draws slightly more from “transformers” than from itself. That is the mechanism by which the blank fills itself from context. With column masking, α_{11} would have been forced to 0, but that would not remove the leak discussed in [Section 2](#).

Problem 5. *BERT-large parameter count, slide method*

Using the slide’s method (vocabulary 30,000, ignore attention biases and LayerNorm, include FFN biases), estimate the parameters of BERT-large ($L = 24$, $H = 1024$, $A = 16$, FFN 4096, $T = 512$).

Solution

Embeddings: token $30000 \times 1024 = 30.72\text{M}$; segment $2 \times 1024 = 2048$; position $512 \times 1024 = 0.524\text{M}$. Total $\approx 31.25\text{M}$.

Per layer: Q, K, V : $(1024 \times 64 \times 3) \times 16 = 3,145,728$ (the same as 3×1024^2). Output $1024^2 = 1,048,576$. FFN: $2 \times 1024 \times 4096 + (4096 + 1024) = 8,388,608 + 5,120 = 8,393,728$. Per layer $\approx 12.59\text{M}$.

24 layers: $24 \times 12.588 = 302.1\text{M}$. **Total** $\approx 31.25 + 302.1 = 333.4\text{M}$, vs 340M in the paper. Note: head dimension = $1024/16 = 64$, the same as base.

Problem 6. *A custom encoder*

You design a small BERT: $|\mathcal{V}| = 32,000$, $T_{\max} = 256$, $d = 512$, 8 heads, FFN 2048, 6 layers, 2 segments. Using the $12Ld^2$ approximation plus embeddings, estimate the parameter count. What is the head dimension?

Solution

Embeddings: $(32000 + 256 + 2) \times 512 = 16,516,096 \approx 16.5\text{M}$. Layers: $12 \times 6 \times 512^2 = 72 \times 262,144 = 18,874,368 \approx 18.9\text{M}$. Total $\approx 35.4\text{M}$. Head dimension = $512/8 = 64$. Here the embedding table is almost half the model, a common feature of small models with large vocabularies.

Problem 7. *Building the input tensors*

Sentence A = “my dog is cute”, sentence B = “he likes play” (one token per word). Maximum length is 12. Write the token sequence, segment ids, position ids and attention (padding) mask.

Solution

pos	0	1	2	3	4	5	6	7	8	9	10	11
token	[CLS]	my	dog	is	cute	[SEP]	he	likes	play	[SEP]	[PAD]	[PAD]
segment	0	0	0	0	0	0	1	1	1	1	0	0
attn mask	1	1	1	1	1	1	1	1	1	1	0	0

10 real tokens and 2 [PAD]. The first [SEP] belongs to segment A; the final [SEP] belongs to segment B. The padding mask adds $-\infty$ to the attention scores of [PAD] *columns*, so no real token attends to padding. (The segment id of [PAD] is irrelevant; 0 is conventional.) The embedding of position 7 is $E_T[\text{likes}] + E_B + E_7$.

Problem 8. *Extractive QA: independent argmax*

A paragraph has 6 tokens. The start scores $S \cdot h_i$ are $(1.0, 0.5, 3.2, 2.1, -0.4, 0.8)$ and the end scores $E \cdot h_i$ are $(0.2, -1.0, 0.6, 1.5, 2.8, 0.1)$. (a) Compute s_i and e_i . (b) What span is predicted? (c) What is the joint score of this span?

Solution

(a) exp of the starts: $(2.718, 1.649, 24.53, 8.166, 0.670, 2.226)$, sum 39.96, so $\mathbf{s} = (0.068, 0.041, \mathbf{0.614}, 0.204, 0.017, 0.056)$. exp of the ends: $(1.221, 0.368, 1.822, 4.482, 16.44, 1.105)$, sum 25.44, so $\mathbf{e} = (0.048, 0.014, 0.072, 0.176, \mathbf{0.646}, 0.043)$. (b) $i^* = 3$, $j^* = 5$. Since $j \geq i$, the answer is tokens 3..5. (c) Joint logit $S \cdot h_3 + E \cdot h_5 = 3.2 + 2.8 = 6.0$. Joint probability $s_3 e_5 = 0.614 \times 0.646 = 0.397$. Checking all valid $(i \leq j)$ pairs confirms that $(3, 5)$ is also the best valid span.

Problem 9. *Extractive QA: when $j < i$*

Same start scores, but now the end scores are (2.9, 0.1, 0.3, 0.5, 0.2, 0.4). What does the slides' rule output? What does "best valid span" decoding output?

Solution

exp of the ends: (18.17, 1.105, 1.350, 1.649, 1.221, 1.492), sum 24.99, so $\mathbf{e} = (0.727, 0.044, 0.054, 0.066, 0.049, 0.060)$, giving $j^* = 1$. The start is still $i^* = 3$, so $j < i$. **Slides' rule:** return the **empty string** (answer not in paragraph). **Best valid span:** maximise $S_i + E_j$ over $i \leq j$. Candidates: (1, 1): $1.0 + 2.9 = 3.9$; (3, 4): $3.2 + 0.5 = 3.7$; (3, 6): 3.6; (3, 3): 3.5. So (1, 1) wins with score 3.9, a one-token answer. This is exactly why SQuAD 2.0 systems compare against a null score at [CLS]: without it, joint decoding always returns *some* span.

Problem 10. *Head sizes*

How many *new* parameters are added for (a) 3-class NLI with a linear head on $h_{[\text{CLS}]}$, with bias; (b) NER with 9 tags, a linear head on every token, with bias; (c) extractive QA with S, E and no bias?

Solution

(a) $3 \times 768 + 3 = \mathbf{2,307}$. (b) $9 \times 768 + 9 = \mathbf{6,921}$. The same head is shared across positions, so the count does not depend on T . (c) $2 \times 768 = \mathbf{1,536}$. Each is tiny next to 110M, yet in fine-tuning all 110M parameters are updated.

Problem 11. *Memory for fine-tuning*

Estimate the GPU memory for the weights, gradients and Adam states when fully fine-tuning BERT-base in fp32. Then estimate the *weights only* in bf16 for inference.

Solution

$N \approx 110\text{M}$. fp32 weights $4N = 440\text{MB}$; gradients $4N = 440\text{MB}$; Adam m, v : $8N = 880\text{MB}$. Total $\approx \mathbf{1.76\text{GB}}$, plus activations, which scale with $\text{batch} \times T \times d \times L$ and with T^2 for attention maps. bf16 inference weights: $2N = \mathbf{220\text{MB}}$.

Problem 12. *Why BERT pre-trained mostly at $T = 128$*

Compare per-layer attention matmul FLOPs ($\approx 4T^2d$) and per-layer projection + FFN FLOPs ($\approx 24Td^2$) at $T = 128$ and $T = 512$ for $d = 768$.

Solution

$T = 128$: attention $4 \cdot 128^2 \cdot 768 = 50.3\text{M}$; dense $24 \cdot 128 \cdot 768^2 = 1812\text{M}$. Attention is $\approx 2.7\%$.
 $T = 512$: attention $4 \cdot 512^2 \cdot 768 = 805\text{M}$; dense $24 \cdot 512 \cdot 768^2 = 7248\text{M}$. Attention is $\approx 10\%$.
 Going $128 \rightarrow 512$ multiplies attention cost by $16\times$ and dense cost by $4\times$. Attention-map *memory* (T^2 per head per layer) also grows $16\times$. Training 90% of steps at 128 saved a lot of compute, and the final 10% at 512 trained the later position embeddings.

Problem 13. *Effective training signal: MLM vs causal LM*

For $T = 512$ and a batch of 256 sequences, how many token predictions contribute to the loss per step for (a) BERT's MLM with 15% masking, (b) a GPT-style causal LM?

Solution

(a) $0.15 \times 512 \times 256 = 19,661$ predictions per step. (b) $512 \times 256 = 131,072$ (every position predicts its next token). The causal LM gets $\approx 6.7\times$ more supervised targets from the same forward pass. MLM pays for bidirectionality with lower sample efficiency, which is the trade-off behind the 15% discussion (and the motivation for ELECTRA's "replaced token detection", which trains on all positions).

6 Subjective questions with model answers

Problem 1.

Why can't a decoder-only transformer be used directly for masked language modelling?

Answer

The decoder uses causal self-attention: position i attends only to positions $\leq i$. MLM needs to predict a masked word from context on *both* sides ("i _ to read"). With a causal mask, the [MASK] position cannot see the right-hand context, so the decoder is inherently unidirectional. The encoder has no causal mask, so it is the natural fit.

Problem 2.

The lecture asks whether we can "mask the attention weights of the words to be masked, as in CLM". Explain in detail why not, and what BERT does instead.

Answer

Masking attention columns stops other tokens from attending to the masked word, but (1) the input vector at that position is still the true word, so the residual connection carries the true identity straight to the output at the same position, and the model could learn to copy it and never learn from context. (2) We want the model to learn, through attention, what the blank should be. That requires the blank to be an actual input symbol whose position gathers context. BERT therefore *corrupts the input* by replacing the chosen words with a learnable [MASK] token (noise) and trains the model to recover them. This is a denoising-autoencoder objective, so MLM is called a pre-training denoising objective.

Problem 3.

Discuss the effect of the masking rate on BERT pre-training.

Answer

Only the masked set $\mathcal{M}$ contributes to the loss. *Very high* masking removes most of the context, so the prediction task becomes nearly impossible and the representations learned are poor. *Very low* masking leaves rich context but gives few targets per sequence, hence small gradient signal per step, slow convergence and inefficient training. BERT uses 15% as a compromise. The optimum depends on model size, masking scheme (token vs span masking) and optimiser, and later work showed larger models can benefit from higher rates such as 40%.

Problem 4.

Explain the 80/10/10 rule and the reasoning behind each part.

Answer

Of the 15% of positions selected: 80% are replaced by [MASK], 10% by a random token, and 10% are left unchanged; all are predicted. The [MASK] token never appears during fine-tuning, so training only on [MASK] would create a pre-train/fine-tune mismatch: the model could learn to produce predictive representations only at [MASK] positions. Random replacement forces the model not to trust any input token blindly and to verify every token against context. Keeping tokens unchanged biases the representation towards the actually observed word, so real tokens get faithful contextual representations. Because the model cannot tell which non-[MASK] tokens are targets, it must represent every token well, which is exactly what fine-tuning (with no masking) needs.

Problem 5.

What is Next Sentence Prediction? Describe how the training data is created and how the prediction is made.

Answer

NSP is a binary classification pre-training task meant to teach inter-sentence relationships (useful for QA, NLI). Pairs (A, B) are formed from the corpus: 50% of the time B is the sentence that actually follows A (IsNext); 50% of the time B is a random corpus sentence (NotNext). The input is [CLS] A [SEP] B [SEP], with segment embeddings E_A/E_B . The final hidden vector of [CLS] is passed through a classifier (softmax over 2 classes) and trained with $\mathcal{L}_{cls} = -\log \hat{y}$. It is trained jointly with MLM: $\mathcal{L} = \mathcal{L}_{MLM} + \mathcal{L}_{cls}$.

Problem 6.

Why does BERT need segment embeddings when it already has the [SEP] token?

Answer

[SEP] marks the boundary only at one position. A token deep inside sentence B would have to infer “I am after the [SEP]” through attention and position arithmetic. A learned segment embedding E_A or E_B is added to *every* token, so sentence membership is directly available at the input of every layer. This makes it easy for attention to distinguish, for example, question tokens from paragraph tokens in QA. It costs only 2×768 parameters.

Problem 7.

Why is the [CLS] token’s final representation used for classification rather than, say, the first word’s?

Answer

[CLS] has no lexical meaning of its own, so its representation is free to become a sequence-level summary. Through bidirectional self-attention in every layer it can gather information from all tokens of both segments. Since the NSP loss (and later the fine-tuning loss) is applied at its output, training explicitly shapes $h_{[CLS]}$ into a useful aggregate. A real word’s vector is also responsible for representing that word (and is used for MLM when masked), so it is a worse summary. Its position (always 0) is also fixed and easy to find.

Problem 8.

Compare the feature-based and fine-tuning approaches for adapting BERT.

Answer

Feature-based: feed the (padded) input to BERT, take the final hidden representation as features, and train a separate head (logistic regression, Naive Bayes, NN). BERT's parameters are frozen and only the head is trained from scratch. Features can be cached and it is cheap, but the representation cannot adapt. **Fine-tuning:** add a randomly initialised head and train the whole model on labelled data, without masking. It converges quickly and needs fewer labelled samples, but it costs a full backward pass and a full model copy per task. Both beat concatenated static word2vec vectors, because BERT features are contextual and composed across the sentence.

Problem 9.

Explain how BERT performs extractive question answering. Include the input format, the learnable parameters, the probability computations, and span selection.

Answer

Input: [CLS] question [SEP] paragraph [SEP], with the question in segment A and the paragraph in segment B. BERT outputs $h_1, \dots, h_n$ for the paragraph tokens. Two learnable vectors $S, E \in \mathbb{R}^d$ are introduced. Start probabilities $s_i = \exp(S \cdot h_i) / \sum_j \exp(S \cdot h_j)$; end probabilities $e_j = \exp(E \cdot h_j) / \sum_i \exp(E \cdot h_i)$. Training maximises $\log s_{i^*} + \log e_{j^*}$ for the gold start and end. At inference, take the most likely start i and end j . If $j \geq i$, output words $i..j$ (e.g. $i = 9, j = 17$: "role of artificial intelligence (AI) in guiding the spacecraft"). If $j < i$, output an empty string, meaning the answer is not in the paragraph.

Problem 10.

BERT predicts masked tokens independently. What does this assumption mean, and what are its consequences?

Answer

BERT models $P(\{y_i\}_{i \in \mathcal{M}} \mid \tilde{\mathbf{x}}) \approx \prod_{i \in \mathcal{M}} P(y_i \mid \tilde{\mathbf{x}})$, so predictions at different masked positions do not condition on each other. Consequence: jointly inconsistent predictions are possible ("New Angeles"), and BERT does not define a proper joint distribution over text, so it cannot simply be sampled from to generate text. For learning representations this is largely harmless, since each position still sees the full unmasked context. Autoregressive models use the exact chain rule instead. XLNet's permutation LM tries to get both benefits.

Problem 11.

Is the MLM objective alone sufficient for tasks like QA? Argue both sides.

Answer

Argument for NSP (the lecture's view): MLM targets are local (a word in its context), and nothing explicitly teaches the relationship between two text segments. QA and NLI require reasoning across segments. Adding NSP was reported to significantly improve QA and NLI in the BERT paper. *Counter-argument (later work):* RoBERTa showed that removing NSP and training longer on contiguous full-length documents matched or beat BERT. With long contiguous inputs, MLM already requires cross-sentence reasoning, and NSP's random negatives are often

solvable by topic cues alone. ALBERT's Sentence Order Prediction is a harder, coherence-focused alternative.

Problem 12.

Where are most of BERT-base's parameters, and how does the number of heads affect the count?

Answer

Embeddings: $\approx 23.4\text{M}$ (mostly the $30\text{K} \times 768$ token table). The 12 layers hold $\approx 84\text{M}$, of which about two thirds ($\approx 4.7\text{M}$ per layer) are in the FFN, while attention has $\approx 2.4\text{M}$ per layer. The number of heads does not change the count: A heads of size d/A give $A \cdot d \cdot (d/A) = d^2$ per projection. Heads only partition the same width into parallel subspaces.

Problem 13.

Why is BERT's approach to pre-training called "denoising"? Relate it to autoencoders.

Answer

A denoising autoencoder receives a corrupted input $\tilde{x}$ (noise added to x) and is trained to reconstruct the clean x , so it must learn the data's structure to undo the noise. In MLM, replacing tokens with [MASK] or random words is the noise, and predicting the original tokens is the reconstruction. Unlike a classical autoencoder, the loss is only on corrupted positions, and there is no bottleneck: the representations at every position are the learned features.

7 Multiple-choice questions

Q1. BERT is built from which part of the transformer?

- (A) Decoder only
- (B) Encoder only
- (C) Encoder and decoder
- (D) Neither; it is an RNN

Answer: B

BERT = Bidirectional *Encoder* Representations from Transformers: a stack of encoder layers.

Q2. Which task is the *most* natural fit for GPT's left-to-right training?

- (A) NER
- (B) Extractive QA
- (C) Text generation
- (D) Fill-in-the-blank

Answer: C

Generation proceeds left to right, so causal LM matches it. The others benefit from right-hand context.

Q3. MLM is most similar in spirit to which word2vec model?

- (A) Skip-gram
- (B) CBOW
- (C) GloVe
- (D) TF-IDF

Answer: B

CBOW predicts a word from its surrounding context words, as MLM predicts a masked word from both sides.

Q4. The MLM loss is averaged over

- (A) all tokens (B) only masked positions $\mathcal{M}$
 (C) only [CLS] (D) only unmasked tokens

Answer: B

$\mathcal{L} = \frac{1}{|\mathcal{M}|} \sum_{y_i \in \mathcal{M}} -\log \hat{y}_i$. Unmasked positions do not enter the loss.

Q5. What fraction of all input tokens are replaced by [MASK] in BERT (in expectation)?

- (A) 15% (B) 12%
 (C) 10% (D) 80%

Answer: B

$0.15 \times 0.80 = 0.12$.

Q6. What fraction of all input tokens are replaced by a random token?

- (A) 10% (B) 1.5%
 (C) 15% (D) 3%

Answer: B

$0.15 \times 0.10 = 0.015$.

Q7. The main reason for not replacing all selected tokens with [MASK] is

- (A) to speed up training
 (B) [MASK] does not appear in downstream (fine-tuning) data
 (C) to reduce vocabulary size
 (D) to make NSP harder

Answer: B

It closes the pre-train/fine-tune mismatch, because fine-tuning inputs contain no [MASK] tokens.

Q8. Why can't we simply mask attention columns to hide the target word?

- (A) Softmax cannot handle $-\infty$
 (B) The true token still enters at its own position and leaks via the residual path; the blank must be an input the model attends from
 (C) It would make the model unidirectional
 (D) It increases parameters

Answer: B

Column masking hides the word from *others*, but not from its own position's output. Corrupting the input is required.

Q9. MLM is also referred to as a

- (A) contrastive objective
- (B) denoising objective
- (C) reinforcement objective
- (D) generative adversarial objective

Answer: B

[MASK] is noise corrupting the input. Recovering the original is denoising, as in denoising autoencoders.

Q10. Very low masking rates mainly cause

- (A) loss of context
- (B) slow convergence due to small gradient signal
- (C) overfitting to [MASK]
- (D) vocabulary explosion

Answer: B

Few targets per sequence means a weak learning signal per step and inefficient training.

Q11. In NSP, the fraction of NotNext examples is

- (A) 10%
- (B) 25%
- (C) 50%
- (D) 80%

Answer: C

50% true next sentence, 50% random sentence.

Q12. For NSP, the prediction is made from

- (A) the average of all token vectors
- (B) the final hidden state of [CLS]
- (C) the final hidden state of [SEP]
- (D) the last token of sentence B

Answer: B

The hidden representation of [CLS] is used for the final classification.

Q13. BERT's input embedding for a token is

- (A) token embedding only
- (B) concatenation of token, segment and position embeddings
- (C) sum of token, segment and position embeddings
- (D) product of token and position embeddings

Answer: C

The three learned embeddings, each in $\mathbb{R}^{768}$, are added.

Q14. The number of parameters in BERT-base's segment embedding table is

- (A) 768 (B) 1,536
(C) 393,216 (D) 23M

Answer: B

$2 \times 768 = 1536$.

Q15. The number of parameters in BERT-base's position embedding table is

- (A) 512×64 (B) $512 \times 768 \approx 0.4\text{M}$
(C) 30000×768 (D) 768×3072

Answer: B

512 positions, each a learned 768-d vector.

Q16. In BERT-base, each attention head's query dimension is

- (A) 768 (B) 128
(C) 64 (D) 12

Answer: C

$768/12 = 64$.

Q17. Per-layer FFN parameters in BERT-base (with biases) are approximately

- (A) 0.6M (B) 1.7M
(C) 4.7M (D) 7M

Answer: C

$768 \cdot 3072 \cdot 2 + 3072 + 768 \approx 4.72\text{M}$.

Q18. Per-layer self-attention Q, K, V weights (no bias) in BERT-base total

- (A) $\approx 0.6\text{M}$ (B) $\approx 1.77\text{M}$
(C) $\approx 2.4\text{M}$ (D) $\approx 7\text{M}$

Answer: B

$(768 \times 64 \times 3) \times 12 = 1,769,472$. With the 0.59M output projection, attention is $\approx 2.36\text{M}$.

Q19. Doubling the number of heads in BERT-base while keeping $d = 768$ changes the attention parameter count by

- (A) $\times 2$ (B) $\times 0.5$
(C) no change (D) $\times 4$

Answer: C

Each head shrinks to d/A . The total per projection stays d^2 .

Q20. BERT-large has

- (A) 12 layers, 768 hidden, 12 heads
- (B) 24 layers, 1024 hidden, 16 heads
- (C) 24 layers, 768 hidden, 16 heads
- (D) 48 layers, 1600 hidden, 25 heads

Answer: B

Large: $L = 24, H = 1024, A = 16$, FFN 4096, $\approx 340\text{M}$ params.

Q21. BERT's pre-training corpora and sizes are

- (A) Wikipedia 800M + BookCorpus 2500M
- (B) BookCorpus 800M + Wikipedia 2500M words
- (C) Common Crawl 1T tokens
- (D) WebText 40GB

Answer: B

BookCorpus 800M words, English Wikipedia 2500M words.

Q22. In the feature-based approach,

- (A) all BERT parameters are fine-tuned
- (B) only the head is trained; BERT is frozen
- (C) the head is frozen and BERT is trained
- (D) inputs are masked at 15%

Answer: B

The body is frozen; the head (LR, NB, NN) is trained from scratch.

Q23. During fine-tuning, input words are

- (A) masked at 15%
- (B) masked at 80/10/10
- (C) not masked
- (D) replaced by random words at 10%

Answer: C

No masking during fine-tuning. That is why pre-training used random and unchanged replacements.

Q24. Compared with feature-based use, fine-tuning is observed to

- (A) need more labelled data
- (B) converge quickly with fewer labelled samples
- (C) never overfit
- (D) require no GPU

Answer: B

As stated in the lecture.

Q25. In extractive QA, the start vector S has dimension

- (A) $|\mathcal{V}|$ (B) number of paragraph tokens
 (C) same as h_i (768) (D) 2

Answer: C S and E are the size of h_i so that $S \cdot h_i$ is a scalar score.**Q26.** The softmax for s_i in extractive QA is taken over

- (A) the vocabulary
 (B) the paragraph token positions
 (C) the two classes IsNext/NotNext
 (D) the attention heads

Answer: B $s_i = \exp(S \cdot h_i) / \sum_j \exp(S \cdot h_j)$ over positions j of the paragraph.**Q27.** If the most probable end index is smaller than the most probable start index, the slides' method returns

- (A) the start token only
 (B) the whole paragraph
 (C) an empty string (answer not in paragraph)
 (D) the [CLS] token

Answer: C $j < i$ is not a valid span, so return the empty string.**Q28.** How many new parameters does the QA head (S, E , no bias) add to BERT-base?

- (A) 768 (B) 1,536
 (C) 2,307 (D) 590,592

Answer: B 2×768 .**Q29.** Why is BERT not typically used to generate text left to right?

- (A) It has no softmax over the vocabulary
 (B) It is trained to predict masked tokens independently using bidirectional context, not a left-to-right chain rule
 (C) It cannot process sequences longer than 10 tokens
 (D) It has no positional embeddings

Answer: B

It does not model $P(x_i | x_{<i})$ and its masked predictions are conditionally independent.

Q30. The approximate total parameter count of BERT-base by the slides' calculation is

- (A) 84M (B) 107.4M
(C) 340M (D) 23.4M

Answer: B

23.4M (embeddings) + 84M (layers) = 107.4M; the paper reports 110M.

Q31. Which statement about the 10% “unchanged” tokens is correct?

- (A) They are excluded from the loss
(B) They are included in the loss and push the representation towards the observed word
(C) They are replaced by [PAD]
(D) They are only used for NSP

Answer: B

All selected positions, unchanged ones included, are prediction targets.

Q32. Which is *deeply* bidirectional?

- (A) ELMo (concatenated forward and backward LSTMs)
(B) GPT
(C) BERT
(D) A unigram LM

Answer: C

BERT mixes left and right context in every layer via self-attention. ELMo only concatenates two unidirectional models.